14. Stream Applications — Quick Notes

Stream Structure

Source → Intermediate Operations → Terminal Operation

Example:

```
list.stream()
  .filter(x -> x % 2 == 0)
  .sorted()
  .forEach(System.out::println);
```

Intermediate Operations

Used For

- filter()
- map()
- sorted()
- distinct()

Example

.filter(x -> x > 10)
Filters: elements greater than 10

Terminal Operations

Used For

- sum
- average
- collect
- print

Example

.forEach(System.out::println)

Why Streams Are Useful?

Clean & Readable Code

Instead of:

- multiple loops
- many if conditions
- break/continue logic

Streams provide: pipeline-based clean code

Old Style vs Stream Style

Old Style

```
for(...) {
  if(...)
}
```

Complex and lengthy.

Stream Style

```
list.stream()
  .filter(...)
  .map(...)
  .collect(...)
```

Readable and concise.

Efficient Processing

Streams are efficient because of: Lazy Evaluation

Important

Elements processed: only when terminal operation runs

Pipeline Processing

Each element: passes through all operations one-by-one
instead of: creating multiple temporary collections

No Extra Storage

Streams: do not have own storage
They are: abstraction over source

Works Great with Lambda

Example: .filter(x -> x % 2 == 0)

Streams heavily use:

- Lambda Expressions
- Method References

Method Reference Example

Instead of: x -> System.out.println(x)

Can write: System.out::println

Example: list.stream().forEach(System.out::println);

Common Stream Applications

Operation	Purpose
filter()	Filtering data
map()	Transforming data
sorted()	Sorting

distinct()	Remove duplicates
collect()	Convert back to collection
forEach()	Printing/processing
count()	Count elements

Parallel Streams

Purpose

Use multiple CPU cores.

Example: list.parallelStream()

Advantage

Faster processing for:

- large datasets
- independent operations

Sequential vs Parallel

Type	Processing
stream()	Sequential
parallelStream()	Parallel

Important Quick Points

Topic	Point
Stream Code	Clean & readable
Lazy Evaluation	Executes at terminal op
Pipeline	Element-by-element processing
Lambda	Works perfectly with streams
Method Reference	Cleaner lambda replacement
Parallel Stream	Multi-core processing
No Storage	Stream is abstraction only